PDF Page Extractor: Complete Code Reference

PDF Page Extractor: Complete Code Reference

This file contains the important project code copied from the current workspace, including constants for routes, messages, status codes, config limits, auth, OTP, JWT, protected PDF APIs, and frontend UI.

backend/package.json


```
{
"name": "backend",
"version": "1.0.0",
"description": "",
"main": "dist/index.js",
"scripts": {
"build": "tsc",
"start": "node dist/index.js",
"dev": "tsx watch index.ts",
"test": "node --import tsx --test \"test/**/*.test.ts\""
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs",
"dependencies": {
"bcryptjs": "^3.0.3",
"cors": "^2.8.6",
"dotenv": "^17.4.2",
"express": "^5.2.1",
"jsonwebtoken": "^9.0.3",
"multer": "^2.1.1",
"node-cron": "^4.2.1",
"nodemailer": "^8.0.7",
"nodemon": "^3.1.14",
"pdf-lib": "^1.17.1",
"uuid": "^14.0.0"
},
"devDependencies": {
"@types/bcryptjs": "^2.4.6",
"@types/cors": "^2.8.19",
"@types/express": "^5.0.6",
"@types/jsonwebtoken": "^9.0.10",
"@types/multer": "^2.1.0",
"@types/node": "^25.7.0",
"@types/node-cron": "^3.0.11",
"@types/nodemailer": "^8.0.0",
"tsx": "^4.21.0",
"typescript": "^6.0.3"
}
}
```


backend/tsconfig.json

```
{
"compilerOptions": {
"target": "ES2022",
"module": "Node16",
"moduleResolution": "Node16",
"rootDir": ".",
"outDir": "dist",
"strict": true,
"esModuleInterop": true,
"forceConsistentCasingInFileNames": true,
"skipLibCheck": true,
"resolveJsonModule": true,
"types": ["node", "multer"]
},
"include": ["index.ts", "src/**/*.ts"],
"exclude": ["node_modules", "dist", "uploads", "outputs"]
}
```

backend/.env.example

```
PORT=5000
NODE_ENV=development
UPLOAD_DIR=uploads
OUTPUT_DIR=outputs
CLEANUP_INTERVAL_MINUTES=60
JWT_SECRET=replace-with-a-long-random-secret
JWT_EXPIRES_IN=7d
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
SMTP_FROM=
```

backend/index.ts

```
import dotenv from 'dotenv';
import app from './src/app';
import { SYSTEM_MESSAGES } from './src/constants/messages';

/**
* ARCHITECTURE: ENTRY POINT
* Purpose: Load environment variables and start the HTTP server.
*/

dotenv.config();

const PORT = Number(process.env.PORT) || 5000;

app.listen(PORT, () => {
console.log(SYSTEM_MESSAGES.SERVER_STARTED.replace('{port}', String(PORT)));
});
```

backend/src/app.ts

```
import fs from 'node:fs';
import path from 'node:path';
import cors from 'cors';
import express, { type ErrorRequestHandler } from 'express';
import { STORAGE } from './constants/config';
import { SYSTEM_MESSAGES } from './constants/messages';
import { API_ROUTES } from './constants/routes';
import { STATUS_CODES } from './constants/statusCodes';
import authRoutes from './routes/authRoutes';
import pdfRoutes from './routes/pdfRoutes';
import setupCleanupJob from './utils/cleanup';

/**
* ARCHITECTURE: APP SETUP
* Purpose: Configure Express middleware, static assets, routes, jobs, and errors.
*/

const app = express();
const uploadDir = path.join(process.cwd(), process.env[STORAGE.UPLOAD_DIR_ENV] ||
STORAGE.DEFAULT_UPLOAD_DIR);
const outputDir = path.join(process.cwd(), process.env[STORAGE.OUTPUT_DIR_ENV] ||
STORAGE.DEFAULT_OUTPUT_DIR);

fs.mkdirSync(uploadDir, { recursive: true });
fs.mkdirSync(outputDir, { recursive: true });

app.use(cors());
app.use(express.json());
app.use(API_ROUTES.OUTPUTS_BASE, express.static(outputDir));
app.use(API_ROUTES.AUTH_BASE, authRoutes);
app.use(API_ROUTES.PDF_BASE, pdfRoutes);

setupCleanupJob(uploadDir, outputDir);

const errorHandler: ErrorRequestHandler = (err, _req, res, next) => {
if (res.headersSent) {
next(err);
return;
}

const message = err instanceof Error ? err.message : SYSTEM_MESSAGES.UNKNOWN_ERROR;
const statusCode = message.includes('PDF')
|| message.includes('page')
|| message.includes('email')
|| message.includes('Password')
|| message.includes('OTP')
|| message.includes('account')
|| message.includes('token')
|| err?.name === 'MulterError'
? STATUS_CODES.BAD_REQUEST
: STATUS_CODES.INTERNAL_SERVER_ERROR;
```

```
res.status(statusCode).json({ error: message });
};

app.use(errorHandler);

export default app;
```

backend/src/constants/config.ts

```
export const STORAGE = {
UPLOAD_DIR_ENV: 'UPLOAD_DIR',
OUTPUT_DIR_ENV: 'OUTPUT_DIR',
DEFAULT_UPLOAD_DIR: 'uploads',
DEFAULT_OUTPUT_DIR: 'outputs',
PDF_MIME_TYPE: 'application/pdf',
PDF_EXTENSION: '.pdf',
EXTRACTED_SUFFIX: '-extracted.pdf'
} as const;

export const AUTH_LIMITS = {
MIN_NAME_LENGTH: 2,
MIN_PASSWORD_LENGTH: 8,
OTP_MIN: 100000,
OTP_MAX: 999999,
OTP_EXPIRY_MINUTES: 10,
OTP_RESEND_COOLDOWN_SECONDS: 60,
MAX_OTP_ATTEMPTS: 5,
BCRYPT_PASSWORD_ROUNDS: 12,
BCRYPT_OTP_ROUNDS: 10,
DEFAULT_JWT_SECRET: 'development-jwt-secret-change-me',
DEFAULT_JWT_EXPIRES_IN: '7d'
} as const;

export const FILE_LIMITS = {
MAX_PDF_SIZE_BYTES: 50 * 1024 * 1024
} as const;

export const SMTP_DEFAULTS = {
PORT: 587,
SECURE_PORT: 465,
SUBJECT: 'Verify your PDF Extractor account'
} as const;

export const CLEANUP = {
DEFAULT_INTERVAL_MINUTES: 60,
CRON_EVERY_HOUR: '0 * * * *'
} as const;
```

backend/src/constants/messages.ts

```
export const AUTH_MESSAGES = {
```

```
NAME_TOO_SHORT: 'Name must be at least 2 characters.',
INVALID_EMAIL: 'Please enter a valid email address.',
PASSWORD_TOO_SHORT: 'Password must be at least 8 characters.',
EMAIL_EXISTS: 'An account already exists with this email.',
SIGNUP_OTP_SENT: 'Signup successful. Please check your email for the OTP.',
SIGNUP_DEV_OTP: 'Signup successful. SMTP is not configured, so use the development OTP from the response.',
ACCOUNT_NOT_FOUND: 'Account not found.',
ACCOUNT_ALREADY_VERIFIED: 'Account is already verified.',
NO_ACTIVE_OTP: 'No active OTP. Please request a new one.',
OTP_EXPIRED: 'OTP expired. Please request a new OTP.',
TOO_MANY_OTP_ATTEMPTS: 'Too many OTP attempts. Please request a new OTP.',
INVALID_OTP: 'Invalid OTP.',
ACCOUNT_VERIFIED: 'Account verified successfully.',
RESEND_OTP_SENT: 'A new OTP has been sent to your email.',
RESEND_DEV_OTP: 'SMTP is not configured, so use the development OTP from the response.',
RESEND_COOLDOWN: 'Please wait {seconds} seconds before requesting another OTP.',
INVALID_CREDENTIALS: 'Invalid email or password.',
VERIFY_BEFORE_LOGIN: 'Please verify your email before logging in.',
LOGIN_SUCCESS: 'Login successful.',
USER_NOT_FOUND: 'User not found.',
TOKEN_REQUIRED: 'Authorization token is required.',
INVALID_TOKEN: 'Invalid or expired token.',
JWT_SECRET_REQUIRED: 'JWT_SECRET must be configured in production.'
} as const;

export const PDF_MESSAGES = {
NO_FILE_UPLOADED: 'Please upload a PDF file.',
INVALID_FILE_TYPE: 'Only PDF files are allowed.',
PDF_NOT_FOUND_REUPLOAD: 'PDF not found. Please upload it again.',
PDF_NOT_FOUND: 'PDF not found.',
PAGES_MUST_BE_ARRAY: 'Pages must be provided as an array.',
PAGES_MUST_BE_WHOLE_NUMBERS: 'Each selected page must be a whole number.',
SELECT_ONE_PAGE: 'Please select at least one page.',
PAGE_INDICES_MUST_BE_WHOLE_NUMBERS: 'Pages must be whole numbers.',
PAGE_OUT_OF_RANGE: 'Page {page} is outside the PDF page range.'
} as const;

export const SYSTEM_MESSAGES = {
UNKNOWN_ERROR: 'Something went wrong.',
UNKNOWN_PDF_ERROR: 'Unknown PDF processing error',
PDF_SERVICE_ERROR_PREFIX: 'PDF Service Error',
CLEANUP_SUCCESS: 'Storage cleanup completed.',
UNKNOWN_CLEANUP_ERROR: 'Unknown cleanup error',
CLEANUP_FAILED_PREFIX: 'Storage cleanup failed',
DEV_OTP_LOG: 'Development OTP for {email}: {otp}',
SERVER_STARTED: 'Server is breathing on http://localhost:{port}'
} as const;
```

backend/src/constants/routes.ts

```
export const API_ROUTES = {
AUTH_BASE: '/api/auth',
PDF_BASE: '/api/pdfs',
```

```
OUTPUTS_BASE: '/outputs'
} as const;

export const AUTH_ROUTES = {
SIGNUP: '/signup',
VERIFY_OTP: '/verify-otp',
RESEND_OTP: '/resend-otp',
LOGIN: '/login',
ME: '/me'
} as const;

export const PDF_ROUTES = {
UPLOAD: '/upload',
BY_ID: '/:id',
EXTRACT: '/:id/extract'
} as const;
```

backend/src/constants/statusCodes.ts

```
export const STATUS_CODES = {
OK: 200,
CREATED: 201,
BAD_REQUEST: 400,
UNAUTHORIZED: 401,
NOT_FOUND: 404,
INTERNAL_SERVER_ERROR: 500
} as const;
```

backend/src/types/models.ts

```
export type PublicUser = {
id: string;
name: string;
email: string;
isVerified: boolean;
createdAt: string;
};

export type UserRecord = PublicUser & {
passwordHash: string;
otpHash: string | null;
otpExpiresAt: string | null;
otpAttempts: number;
otpLastSentAt: string | null;
};

export type PdfRecord = {
id: string;
userId: string;
originalName: string;
size: number;
pageCount: number;
```

```
path: string;
createdAt: string;
};

export type AuthTokenPayload = {
userId: string;
email: string;
};
```

backend/src/repositories/jsonFileRepository.ts

```
import fs from 'node:fs/promises';
import path from 'node:path';

export class JsonFileRepository<TRecord extends { id: string }> {
constructor(private readonly filePath: string) {}

private async ensureFile(): Promise<void> {
await fs.mkdir(path.dirname(this.filePath), { recursive: true });

try {
await fs.access(this.filePath);
} catch {
await fs.writeFile(this.filePath, '[]');
}
}

async findAll(): Promise<TRecord[]> {
await this.ensureFile();
const content = await fs.readFile(this.filePath, 'utf8');
return JSON.parse(content) as TRecord[];
}

async findById(id: string): Promise<TRecord | null> {
const records = await this.findAll();
return records.find((record) => record.id === id) ?? null;
}

async save(record: TRecord): Promise<TRecord> {
const records = await this.findAll();
const existingIndex = records.findIndex((item) => item.id === record.id);

if (existingIndex === -1) {
records.push(record);
} else {
records[existingIndex] = record;
}

await fs.writeFile(this.filePath, JSON.stringify(records, null, 2));
return record;
}
}
```

backend/src/repositories/userRepository.ts

```
import path from 'node:path';
import { JsonFileRepository } from './jsonFileRepository';
import type { UserRecord } from '../types/models';

class UserRepository extends JsonFileRepository<UserRecord> {
constructor() {
super(path.join(process.cwd(), 'data', 'users.json'));
}

async findByEmail(email: string): Promise<UserRecord | null> {
const users = await this.findAll();
return users.find((user) => user.email.toLowerCase() === email.toLowerCase()) ?? null;
}
}

export default new UserRepository();
```

backend/src/repositories/pdfRepository.ts

```
import path from 'node:path';
import { JsonFileRepository } from './jsonFileRepository';
import type { PdfRecord } from '../types/models';

class PdfRepository extends JsonFileRepository<PdfRecord> {
constructor() {
super(path.join(process.cwd(), 'data', 'pdfs.json'));
}

async findOwnedByUser(id: string, userId: string): Promise<PdfRecord | null> {
const pdf = await this.findById(id);

if (!pdf || pdf.userId !== userId) {
return null;
}

return pdf;
}
}

export default new PdfRepository();
```

backend/src/middleware/authenticate.ts

```
import type { NextFunction, Request, Response } from 'express';
import { AUTH_MESSAGES } from '../constants/messages';
import { STATUS_CODES } from '../constants/statusCodes';
import authService from '../services/authService';
import type { AuthTokenPayload } from '../types/models';
```

```
export type AuthenticatedRequest<
TParams = Record<string, string>,
TResBody = unknown,
TReqBody = unknown
> = Request<TParams, TResBody, TReqBody> & {
user: AuthTokenPayload;
};

export const authenticate = (req: Request, res: Response, next: NextFunction): void => {
const authHeader = req.headers.authorization;

if (!authHeader?.startsWith('Bearer ')) {
res.status(STATUS_CODES.UNAUTHORIZED).json({ error: AUTH_MESSAGES.TOKEN_REQUIRED });
return;
}

try {
const token = authHeader.replace('Bearer ', '').trim();
const payload = authService.verifyToken(token);
(req as AuthenticatedRequest).user = payload;
next();
} catch {
res.status(STATUS_CODES.UNAUTHORIZED).json({ error: AUTH_MESSAGES.INVALID_TOKEN });
}
};
```

backend/src/services/authService.ts

```
import crypto from 'node:crypto';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import type { SignOptions } from 'jsonwebtoken';
import { AUTH_LIMITS } from '../constants/config';
import { AUTH_MESSAGES } from '../constants/messages';
import emailService from './emailService';
import userRepository from '../repositories/userRepository';
import type { AuthTokenPayload, PublicUser, UserRecord } from '../types/models';

type SignupInput = {
name: string;
email: string;
password: string;
};

type AuthResponse = {
message: string;
user: PublicUser;
token?: string;
devOtp?: string;
};

const toPublicUser = (user: UserRecord): PublicUser => ({
id: user.id,
```

```
name: user.name,
email: user.email,
isVerified: user.isVerified,
createdAt: user.createdAt
});

const getJwtSecret = (): string => {
const secret = process.env.JWT_SECRET;

if (!secret && process.env.NODE_ENV === 'production') {
throw new Error(AUTH_MESSAGES.JWT_SECRET_REQUIRED);
}

return secret || AUTH_LIMITS.DEFAULT_JWT_SECRET;
};

const normalizeEmail = (email: string): string => email.trim().toLowerCase();

const generateOtp = (): string => crypto.randomInt(AUTH_LIMITS.OTP_MIN, AUTH_LIMITS.OTP_MAX).toString();

class AuthService {
async signup(input: SignupInput): Promise<AuthResponse> {
const name = input.name.trim();
const email = normalizeEmail(input.email);
const password = input.password;

if (name.length < AUTH_LIMITS.MIN_NAME_LENGTH) {
throw new Error(AUTH_MESSAGES.NAME_TOO_SHORT);
}

if (!/^\S+@\S+\.\S+$/.test(email)) {
throw new Error(AUTH_MESSAGES.INVALID_EMAIL);
}

if (password.length < AUTH_LIMITS.MIN_PASSWORD_LENGTH) {
throw new Error(AUTH_MESSAGES.PASSWORD_TOO_SHORT);
}

const existingUser = await userRepository.findByEmail(email);

if (existingUser?.isVerified) {
throw new Error(AUTH_MESSAGES.EMAIL_EXISTS);
}

const otp = generateOtp();
const otpHash = await bcrypt.hash(otp, AUTH_LIMITS.BCRYPT_OTP_ROUNDS);
const passwordHash = await bcrypt.hash(password, AUTH_LIMITS.BCRYPT_PASSWORD_ROUNDS);
const now = new Date();
const otpExpiresAt = new Date(now.getTime() + AUTH_LIMITS.OTP_EXPIRY_MINUTES * 60 * 1000).toISOString();

const user: UserRecord = {
id: existingUser?.id || crypto.randomUUID(),
name,
email,
```

```
passwordHash,
isVerified: false,
otpHash,
otpExpiresAt,
otpAttempts: 0,
otpLastSentAt: now.toISOString(),
createdAt: existingUser?.createdAt || now.toISOString()
};

await userRepository.save(user);
const emailResult = await emailService.sendOtp(email, otp);

return {
message: emailResult.delivered
? AUTH_MESSAGES.SIGNUP_OTP_SENT
: AUTH_MESSAGES.SIGNUP_DEV_OTP,
user: toPublicUser(user),
devOtp: emailResult.devMode && process.env.NODE_ENV !== 'production' ? otp : undefined
};
}

async verifyOtp(emailInput: string, otpInput: string): Promise<AuthResponse> {
const email = normalizeEmail(emailInput);
const otp = otpInput.trim();
const user = await userRepository.findByEmail(email);

if (!user) {
throw new Error(AUTH_MESSAGES.ACCOUNT_NOT_FOUND);
}

if (user.isVerified) {
return {
message: AUTH_MESSAGES.ACCOUNT_ALREADY_VERIFIED,
user: toPublicUser(user),
token: this.signToken(user)
};
}

if (!user.otpHash || !user.otpExpiresAt) {
throw new Error(AUTH_MESSAGES.NO_ACTIVE_OTP);
}

if (new Date(user.otpExpiresAt).getTime() < Date.now()) {
throw new Error(AUTH_MESSAGES.OTP_EXPIRED);
}

if (user.otpAttempts >= AUTH_LIMITS.MAX_OTP_ATTEMPTS) {
throw new Error(AUTH_MESSAGES.TOO_MANY_OTP_ATTEMPTS);
}

const isOtpValid = await bcrypt.compare(otp, user.otpHash);

if (!isOtpValid) {
user.otpAttempts += 1;
```

```
await userRepository.save(user);
throw new Error(${AUTH_MESSAGES.INVALID_OTP} ${AUTH_LIMITS.MAX_OTP_ATTEMPTS - user.otpAttempts}
attempts left.);
}

user.isVerified = true;
user.otpHash = null;
user.otpExpiresAt = null;
user.otpAttempts = 0;
user.otpLastSentAt = null;
await userRepository.save(user);

return {
message: AUTH_MESSAGES.ACCOUNT_VERIFIED,
user: toPublicUser(user),
token: this.signToken(user)
};
}

async resendOtp(emailInput: string): Promise<AuthResponse> {
const email = normalizeEmail(emailInput);
const user = await userRepository.findByEmail(email);

if (!user) {
throw new Error(AUTH_MESSAGES.ACCOUNT_NOT_FOUND);
}

if (user.isVerified) {
throw new Error(AUTH_MESSAGES.ACCOUNT_ALREADY_VERIFIED);
}

if (user.otpLastSentAt) {
const secondsSinceLastSend = (Date.now() - new Date(user.otpLastSentAt).getTime()) / 1000;

if (secondsSinceLastSend < AUTH_LIMITS.OTP_RESEND_COOLDOWN_SECONDS) {
throw new Error(
AUTH_MESSAGES.RESEND_COOLDOWN.replace(
'{seconds}',
String(Math.ceil(AUTH_LIMITS.OTP_RESEND_COOLDOWN_SECONDS - secondsSinceLastSend))
)
);
}
}

const otp = generateOtp();
user.otpHash = await bcrypt.hash(otp, AUTH_LIMITS.BCRYPT_OTP_ROUNDS);
user.otpExpiresAt = new Date(Date.now() + AUTH_LIMITS.OTP_EXPIRY_MINUTES * 60 * 1000).toISOString();
user.otpAttempts = 0;
user.otpLastSentAt = new Date().toISOString();
await userRepository.save(user);

const emailResult = await emailService.sendOtp(email, otp);

return {
```

```
message: emailResult.delivered
? AUTH_MESSAGES.RESEND_OTP_SENT
: AUTH_MESSAGES.RESEND_DEV_OTP,
user: toPublicUser(user),
devOtp: emailResult.devMode && process.env.NODE_ENV !== 'production' ? otp : undefined
};
}

async login(emailInput: string, password: string): Promise<AuthResponse> {
const email = normalizeEmail(emailInput);
const user = await userRepository.findByEmail(email);

if (!user) {
throw new Error(AUTH_MESSAGES.INVALID_CREDENTIALS);
}

const isPasswordValid = await bcrypt.compare(password, user.passwordHash);

if (!isPasswordValid) {
throw new Error(AUTH_MESSAGES.INVALID_CREDENTIALS);
}

if (!user.isVerified) {
throw new Error(AUTH_MESSAGES.VERIFY_BEFORE_LOGIN);
}

return {
message: AUTH_MESSAGES.LOGIN_SUCCESS,
user: toPublicUser(user),
token: this.signToken(user)
};
}

async getUserById(userId: string): Promise<PublicUser | null> {
const user = await userRepository.findById(userId);
return user ? toPublicUser(user) : null;
}

verifyToken(token: string): AuthTokenPayload {
return jwt.verify(token, getJwtSecret()) as AuthTokenPayload;
}

private signToken(user: UserRecord): string {
const payload: AuthTokenPayload = {
userId: user.id,
email: user.email
};

const expiresIn = (process.env.JWT_EXPIRES_IN || AUTH_LIMITS.DEFAULT_JWT_EXPIRES_IN) as
SignOptions['expiresIn'];
return jwt.sign(payload, getJwtSecret(), { expiresIn });
}
}
```

```
export default new AuthService();
```

backend/src/services/emailService.ts

```
import nodemailer from 'nodemailer';
import { SMTP_DEFAULTS } from '../constants/config';
import { SYSTEM_MESSAGES } from '../constants/messages';

type SendOtpResult = {
delivered: boolean;
devMode: boolean;
};

class EmailService {
async sendOtp(email: string, otp: string): Promise<SendOtpResult> {
const host = process.env.SMTP_HOST;
const port = Number(process.env.SMTP_PORT || SMTP_DEFAULTS.PORT);
const user = process.env.SMTP_USER;
const pass = process.env.SMTP_PASS;
const from = process.env.SMTP_FROM || user;

if (!host || !user || !pass || !from) {
console.log(SYSTEM_MESSAGES.DEV_OTP_LOG.replace('{email}', email).replace('{otp}', otp));
return { delivered: false, devMode: true };
}

const transporter = nodemailer.createTransport({
host,
port,
secure: port === SMTP_DEFAULTS.SECURE_PORT,
auth: { user, pass }
});

await transporter.sendMail({
from,
to: email,
subject: SMTP_DEFAULTS.SUBJECT,
text: Your verification OTP is ${otp}. It expires in 10 minutes.,
html: <p>Your verification OTP is <strong>${otp}</strong>.</p><p>It expires in 10 minutes.</p>
});

return { delivered: true, devMode: false };
}
}

export default new EmailService();
```

backend/src/services/pdfService.ts

```
import fs from 'node:fs/promises';
import { PDFDocument } from 'pdf-lib';
import { PDF_MESSAGES, SYSTEM_MESSAGES } from '../constants/messages';
```

```
type PdfMetadata = {
pageCount: number;
};

/**
* ARCHITECTURE: SERVICE LAYER
* Purpose: Keep PDF business logic independent from Express HTTP details.
*/
class PdfService {
/**
* Extracts selected pages and creates a new PDF.
* pageIndices are zero-based because pdf-lib uses zero-based page positions.
*/
async extractPages(sourcePath: string, pageIndices: number[]): Promise<Buffer> {
try {
await this.validatePageRange(sourcePath, pageIndices);

const sourceBytes = await fs.readFile(sourcePath);
const sourcePdf = await PDFDocument.load(sourceBytes);
const outputPdf = await PDFDocument.create();
const copiedPages = await outputPdf.copyPages(sourcePdf, pageIndices);

copiedPages.forEach((page) => outputPdf.addPage(page));

const pdfBytes = await outputPdf.save();
return Buffer.from(pdfBytes);
} catch (error) {
const message = error instanceof Error ? error.message : SYSTEM_MESSAGES.UNKNOWN_PDF_ERROR;
throw new Error(${SYSTEM_MESSAGES.PDF_SERVICE_ERROR_PREFIX}: ${message});
}
}

async validatePageRange(sourcePath: string, pageIndices: number[]): Promise<PdfMetadata> {
if (!Array.isArray(pageIndices) || pageIndices.length === 0) {
throw new Error(PDF_MESSAGES.SELECT_ONE_PAGE);
}

if (!pageIndices.every(Number.isInteger)) {
throw new Error(PDF_MESSAGES.PAGE_INDICES_MUST_BE_WHOLE_NUMBERS);
}

const sourceBytes = await fs.readFile(sourcePath);
const sourcePdf = await PDFDocument.load(sourceBytes);
const pageCount = sourcePdf.getPageCount();
const outOfRangePage = pageIndices.find((pageIndex) => pageIndex < 0 || pageIndex >= pageCount);

if (outOfRangePage !== undefined) {
throw new Error(PDF_MESSAGES.PAGE_OUT_OF_RANGE.replace('{page}', String(outOfRangePage + 1)));
}

return { pageCount };
}
```

```
async getMetadata(sourcePath: string): Promise<PdfMetadata> {
const sourceBytes = await fs.readFile(sourcePath);
const sourcePdf = await PDFDocument.load(sourceBytes);

return {
pageCount: sourcePdf.getPageCount()
};
}
}

export default new PdfService();
```

backend/src/controllers/authController.ts

```
import type { NextFunction, Request, Response } from 'express';
import { AUTH_MESSAGES } from '../constants/messages';
import { STATUS_CODES } from '../constants/statusCodes';
import authService from '../services/authService';
import type { AuthenticatedRequest } from '../middleware/authenticate';

export const signup = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
try {
const result = await authService.signup(req.body);
res.status(STATUS_CODES.CREATED).json(result);
} catch (error) {
next(error);
}
};

export const verifyOtp = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
try {
const result = await authService.verifyOtp(String(req.body.email || ''), String(req.body.otp || ''));
res.status(STATUS_CODES.OK).json(result);
} catch (error) {
next(error);
}
};

export const resendOtp = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
try {
const result = await authService.resendOtp(String(req.body.email || ''));
res.status(STATUS_CODES.OK).json(result);
} catch (error) {
next(error);
}
};

export const login = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
try {
const result = await authService.login(String(req.body.email || ''), String(req.body.password || ''));
res.status(STATUS_CODES.OK).json(result);
} catch (error) {
next(error);
```

```
}
};

export const me = async (req: AuthenticatedRequest, res: Response, next: NextFunction): Promise<void> => {
try {
const user = await authService.getUserById(req.user.userId);

if (!user) {
res.status(STATUS_CODES.NOT_FOUND).json({ error: AUTH_MESSAGES.USER_NOT_FOUND });
return;
}

res.status(STATUS_CODES.OK).json({ user });
} catch (error) {
next(error);
}
};
```

backend/src/controllers/pdfController.ts

```
import crypto from 'node:crypto';
import fs from 'node:fs/promises';
import path from 'node:path';
import type { NextFunction, Response } from 'express';
import { STORAGE } from '../constants/config';
import { PDF_MESSAGES } from '../constants/messages';
import { API_ROUTES } from '../constants/routes';
import { STATUS_CODES } from '../constants/statusCodes';
import pdfRepository from '../repositories/pdfRepository';
import pdfService from '../services/pdfService';
import type { AuthenticatedRequest } from '../middleware/authenticate';

/**
* ARCHITECTURE: CONTROLLER LAYER
* Purpose: Validate HTTP input, call services, and shape HTTP responses.
*/

type ExtractRequestBody = {
pages?: unknown;
};

const getOutputPath = (id: string): string => path.join(process.cwd(), process.env[STORAGE.OUTPUT_DIR_ENV] ||
STORAGE.DEFAULT_OUTPUT_DIR, id);

const fileExists = async (filePath: string): Promise<boolean> => {
try {
await fs.access(filePath);
return true;
} catch {
return false;
}
};
```

```
const toPageIndices = (pages: unknown): number[] => {
if (!Array.isArray(pages)) {
throw new Error(PDF_MESSAGES.PAGES_MUST_BE_ARRAY);
}

return pages.map((page) => {
const pageNumber = Number(page);

if (!Number.isInteger(pageNumber)) {
throw new Error(PDF_MESSAGES.PAGES_MUST_BE_WHOLE_NUMBERS);
}

return pageNumber - 1;
});
};

export const uploadPdf = async (req: AuthenticatedRequest, res: Response, next: NextFunction): Promise<void> => {
try {
if (!req.file) {
res.status(STATUS_CODES.BAD_REQUEST).json({ error: PDF_MESSAGES.NO_FILE_UPLOADED });
return;
}

const metadata = await pdfService.getMetadata(req.file.path);

await pdfRepository.save({
id: req.file.filename,
userId: req.user.userId,
originalName: req.file.originalname,
size: req.file.size,
pageCount: metadata.pageCount,
path: req.file.path,
createdAt: new Date().toISOString()
});

res.status(STATUS_CODES.CREATED).json({
id: req.file.filename,
name: req.file.originalname,
size: req.file.size,
pageCount: metadata.pageCount,
previewUrl: ${API_ROUTES.PDF_BASE}/${req.file.filename}
});
} catch (error) {
next(error);
}
};

export const extractPdfPages = async (
req: AuthenticatedRequest<{ id: string }, unknown, ExtractRequestBody>,
res: Response,
next: NextFunction
): Promise<void> => {
try {
const { id } = req.params;
```

```
const pdfRecord = await pdfRepository.findOwnedByUser(id, req.user.userId);

if (!pdfRecord || !(await fileExists(pdfRecord.path))) {
res.status(STATUS_CODES.NOT_FOUND).json({ error: PDF_MESSAGES.PDF_NOT_FOUND_REUPLOAD });
return;
}

const pageIndices = toPageIndices(req.body.pages);
const extractedPdf = await pdfService.extractPages(pdfRecord.path, pageIndices);
const outputFileName = ${crypto.randomUUID()}${STORAGE.EXTRACTED_SUFFIX};
const outputPath = getOutputPath(outputFileName);

await fs.mkdir(path.dirname(outputPath), { recursive: true });
await fs.writeFile(outputPath, extractedPdf);

res.status(STATUS_CODES.CREATED).json({
fileName: outputFileName,
pageCount: pageIndices.length,
downloadUrl: ${API_ROUTES.OUTPUTS_BASE}/${outputFileName}
});
} catch (error) {
next(error);
}
};

export const getPdf = async (
req: AuthenticatedRequest<{ id: string }>,
res: Response,
next: NextFunction
): Promise<void> => {
try {
const pdfRecord = await pdfRepository.findOwnedByUser(req.params.id, req.user.userId);

if (!pdfRecord || !(await fileExists(pdfRecord.path))) {
res.status(STATUS_CODES.NOT_FOUND).json({ error: PDF_MESSAGES.PDF_NOT_FOUND });
return;
}

res.setHeader('Content-Type', 'application/pdf');
res.setHeader('Content-Disposition', 'inline');
res.sendFile(path.resolve(pdfRecord.path));
} catch (error) {
next(error);
}
};
```

backend/src/config/multer.ts

```
import crypto from 'node:crypto';
import fs from 'node:fs';
import path from 'node:path';
import multer from 'multer';
import { FILE_LIMITS, STORAGE } from '../constants/config';
```

```
import { PDF_MESSAGES } from '../constants/messages';

/**
* ARCHITECTURE: CONFIG LAYER
* Purpose: Centralize upload storage, limits, and file validation.
*/

const storage = multer.diskStorage({
destination: (_req, _file, cb) => {
const uploadDir = process.env[STORAGE.UPLOAD_DIR_ENV] || STORAGE.DEFAULT_UPLOAD_DIR;
fs.mkdirSync(uploadDir, { recursive: true });
cb(null, uploadDir);
},
filename: (_req, file, cb) => {
cb(null, ${crypto.randomUUID()}${path.extname(file.originalname).toLowerCase()});
}
});

const fileFilter: multer.Options['fileFilter'] = (_req, file, cb) => {
const isPdfMime = file.mimetype === STORAGE.PDF_MIME_TYPE;
const isPdfExtension = path.extname(file.originalname).toLowerCase() === STORAGE.PDF_EXTENSION;

if (isPdfMime && isPdfExtension) {
cb(null, true);
return;
}

cb(new Error(PDF_MESSAGES.INVALID_FILE_TYPE));
};

const upload = multer({
storage,
fileFilter,
limits: { fileSize: FILE_LIMITS.MAX_PDF_SIZE_BYTES }
});

export default upload;
```

backend/src/routes/authRoutes.ts

```
import { Router, type RequestHandler } from 'express';
import { AUTH_ROUTES } from '../constants/routes';
import { login, me, resendOtp, signup, verifyOtp } from '../controllers/authController';
import { authenticate } from '../middleware/authenticate';

const router = Router();

router.post(AUTH_ROUTES.SIGNUP, signup);
router.post(AUTH_ROUTES.VERIFY_OTP, verifyOtp);
router.post(AUTH_ROUTES.RESEND_OTP, resendOtp);
router.post(AUTH_ROUTES.LOGIN, login);
router.get(AUTH_ROUTES.ME, authenticate, me as RequestHandler);
```

```
export default router;
```

backend/src/routes/pdfRoutes.ts

```
import { Router, type RequestHandler } from 'express';
import { PDF_ROUTES } from '../constants/routes';
import upload from '../config/multer';
import { extractPdfPages, getPdf, uploadPdf } from '../controllers/pdfController';
import { authenticate } from '../middleware/authenticate';

/**
* ARCHITECTURE: ROUTES LAYER
* Purpose: Map endpoint paths and HTTP verbs to controller functions.
*/

const router = Router();

router.post(PDF_ROUTES.UPLOAD, authenticate, upload.single('pdf'), uploadPdf as RequestHandler);
router.get(PDF_ROUTES.BY_ID, authenticate, getPdf as RequestHandler);
router.post(PDF_ROUTES.EXTRACT, authenticate, extractPdfPages as RequestHandler);

export default router;
```

backend/src/utils/cleanup.ts

```
import fs from 'node:fs';
import path from 'node:path';
import cron from 'node-cron';
import { CLEANUP } from '../constants/config';
import { SYSTEM_MESSAGES } from '../constants/messages';

/**
* ARCHITECTURE: UTILS / JOBS
* Purpose: Remove old uploaded/generated files so server storage does not grow forever.
*/

const setupCleanupJob = (uploadDir: string, outputDir: string): void => {
const maxAgeMinutes = Number(process.env.CLEANUP_INTERVAL_MINUTES ||
CLEANUP.DEFAULT_INTERVAL_MINUTES);
const maxAgeMs = maxAgeMinutes * 60 * 1000;

const deleteOldFiles = (directory: string): void => {
fs.mkdirSync(directory, { recursive: true });

for (const fileName of fs.readdirSync(directory)) {
const filePath = path.join(directory, fileName);
const fileStats = fs.statSync(filePath);
const ageMs = Date.now() - fileStats.mtimeMs;

if (fileStats.isFile() && ageMs > maxAgeMs) {
fs.unlinkSync(filePath);
}
```

```
}
};

cron.schedule(CLEANUP.CRON_EVERY_HOUR, () => {
try {
deleteOldFiles(uploadDir);
deleteOldFiles(outputDir);
console.log(SYSTEM_MESSAGES.CLEANUP_SUCCESS);
} catch (error) {
const message = error instanceof Error ? error.message : SYSTEM_MESSAGES.UNKNOWN_CLEANUP_ERROR;
console.error(${SYSTEM_MESSAGES.CLEANUP_FAILED_PREFIX}: ${message});
}
});
};

export default setupCleanupJob;
```

backend/test/pdfService.test.ts

```
import assert from 'node:assert/strict';
import fs from 'node:fs/promises';
import os from 'node:os';
import path from 'node:path';
import test from 'node:test';
import { PDFDocument } from 'pdf-lib';
import pdfService from '../src/services/pdfService';

const createSamplePdf = async (): Promise<string> => {
const pdf = await PDFDocument.create();
pdf.addPage([300, 300]);
pdf.addPage([300, 300]);
pdf.addPage([300, 300]);

const filePath = path.join(os.tmpdir(), sample-${Date.now()}.pdf);
await fs.writeFile(filePath, await pdf.save());
return filePath;
};

test('extractPages creates a new PDF with selected pages in requested order', async () => {
const sourcePath = await createSamplePdf();
const outputBuffer = await pdfService.extractPages(sourcePath, [2, 0]);
const outputPdf = await PDFDocument.load(outputBuffer);

assert.equal(outputPdf.getPageCount(), 2);
});

test('extractPages rejects pages outside the source PDF range', async () => {
const sourcePath = await createSamplePdf();

await assert.rejects(
() => pdfService.extractPages(sourcePath, [5]),
/outside the PDF page range/
);
```

```
});
```

frontend/package.json

```
{
"name": "frontend",
"private": true,
"version": "0.0.0",
"type": "module",
"scripts": {
"dev": "vite",
"build": "tsc -b && vite build",
"lint": "eslint .",
"preview": "vite preview"
},
"dependencies": {
"axios": "^1.16.0",
"framer-motion": "^12.38.0",
"lucide-react": "^1.14.0",
"pdfjs-dist": "^5.7.284",
"react": "^19.2.6",
"react-dom": "^19.2.6",
"react-dropzone": "^15.0.0"
},
"devDependencies": {
"@eslint/js": "^10.0.1",
"@types/node": "^24.12.3",
"@types/react": "^19.2.14",
"@types/react-dom": "^19.2.3",
"@vitejs/plugin-react": "^6.0.1",
"eslint": "^10.3.0",
"eslint-plugin-react-hooks": "^7.1.1",
"eslint-plugin-react-refresh": "^0.5.2",
"globals": "^17.6.0",
"typescript": "~6.0.2",
"typescript-eslint": "^8.59.2",
"vite": "^8.0.12"
}
}
```

frontend/.env.example

```
VITE_API_URL=http://localhost:5000
```

frontend/src/constants/api.ts

```
export const API_BASE_URL = import.meta.env.VITE_API_URL ?? 'http://localhost:5000';

export const API_ENDPOINTS = {
AUTH: {
SIGNUP: '/api/auth/signup',
```

```
VERIFY_OTP: '/api/auth/verify-otp',
RESEND_OTP: '/api/auth/resend-otp',
LOGIN: '/api/auth/login',
ME: '/api/auth/me',
},
PDFS: {
UPLOAD: '/api/pdfs/upload',
EXTRACT: (id: string) => /api/pdfs/${id}/extract,
},
} as const;

export const STORAGE_KEYS = {
TOKEN: 'pdf_extractor_token',
} as const;

export const HTTP_HEADERS = {
AUTHORIZATION: 'Authorization',
BEARER: 'Bearer',
} as const;
```

frontend/src/constants/messages.ts

```
export const UI_MESSAGES = {
REFRESH_AND_RETRY: 'Refresh the page and try your PDF again.',
SIGNUP_FAILED: 'Signup failed.',
OTP_VERIFICATION_FAILED: 'OTP verification failed.',
RESEND_OTP_FAILED: 'Could not resend OTP.',
LOGIN_FAILED: 'Login failed.',
INVALID_PDF: 'Please choose a valid PDF file.',
UPLOAD_FAILED: 'Upload failed.',
UPLOAD_UNEXPECTED_RESPONSE: 'Upload returned an unexpected response.',
PDF_UPLOADED: 'PDF uploaded. Select the pages you want.',
BACKEND_UNREACHABLE: 'Could not reach the backend server.',
SELECT_PAGE_FIRST: 'Select at least one page first.',
EXTRACTION_SUCCESS: 'New PDF created. Download is ready.',
EXTRACTION_FAILED: 'Extraction failed.',
} as const;
```

frontend/src/main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
<StrictMode>
<App />
</StrictMode>,
)
```

frontend/src/App.tsx

```
import { Component, type ChangeEvent, type DragEvent, type ErrorInfo, type ReactNode, useEffect, useRef, useState }
from 'react'
import axios from 'axios'
import { motion } from 'framer-motion'
import * as pdfjsLib from 'pdfjs-dist'
import { ArrowDown, ArrowUp, Check, Download, FileText, Loader2, LogOut, RotateCcw, Scissors, ShieldCheck,
UploadCloud, X } from 'lucide-react'
import { API_BASE_URL, API_ENDPOINTS, HTTP_HEADERS, STORAGE_KEYS } from './constants/api'
import { UI_MESSAGES } from './constants/messages'
import './App.css'

pdfjsLib.GlobalWorkerOptions.workerSrc = new URL(
'pdfjs-dist/build/pdf.worker.mjs',
import.meta.url,
).toString()

type UploadedPdf = {
id: string
name: string
size: number
pageCount: number
previewUrl: string
}

type User = {
id: string
name: string
email: string
isVerified: boolean
createdAt: string
}

type ExtractedPdf = {
fileName: string
pageCount: number
downloadUrl: string
}

type ToastState = {
tone: 'success' | 'error'
message: string
}

type AuthMode = 'login' | 'signup' | 'verify'

type ErrorBoundaryState = {
hasError: boolean
}

class ErrorBoundary extends Component<{ children: ReactNode }, ErrorBoundaryState> {
state: ErrorBoundaryState = { hasError: false }
```

```
static getDerivedStateFromError() {
return { hasError: true }
}

componentDidCatch(error: Error, errorInfo: ErrorInfo) {
console.error(error, errorInfo)
}

render() {
if (this.state.hasError) {
return (
<main className="app-shell">
<section className="empty-state">
<FileText size={42} />
<h1>Something went wrong</h1>
<p>{UI_MESSAGES.REFRESH_AND_RETRY}</p>
</section>
</main>
)
}

return this.props.children
}
}

const formatFileSize = (bytes: number) => {
if (bytes < 1024 * 1024) {
return ${Math.max(1, Math.round(bytes / 1024))} KB
}

return ${(bytes / 1024 / 1024).toFixed(1)} MB
}

const getAssetUrl = (path: string) => {
if (path.startsWith('http')) {
return path
}

return ${API_BASE_URL}${path}
}

function PageCard({
pdfUrl,
token,
pageNumber,
selectedOrder,
onToggle,
onMove,
}: {
pdfUrl: string
token: string
pageNumber: number
selectedOrder: number | null
onToggle: (pageNumber: number) => void
```

```
onMove: (pageNumber: number, direction: -1 | 1) => void
}) {
const canvasRef = useRef<HTMLCanvasElement | null>(null)

useEffect(() => {
let cancelled = false

const renderPage = async () => {
const canvas = canvasRef.current
const context = canvas?.getContext('2d')

if (!canvas || !context) {
return
}

const loadingTask = pdfjsLib.getDocument({
url: pdfUrl,
httpHeaders: {
[HTTP_HEADERS.AUTHORIZATION]: ${HTTP_HEADERS.BEARER} ${token},
},
})
const pdf = await loadingTask.promise
const page = await pdf.getPage(pageNumber)
const viewport = page.getViewport({ scale: 0.45 })

if (cancelled) {
return
}

canvas.width = viewport.width
canvas.height = viewport.height
await page.render({ canvas, canvasContext: context, viewport }).promise
}

renderPage().catch(() => {
const canvas = canvasRef.current
const context = canvas?.getContext('2d')

if (canvas && context) {
context.fillStyle = '#151926'
context.fillRect(0, 0, canvas.width || 220, canvas.height || 300)
}
})

return () => {
cancelled = true
}
}, [pdfUrl, pageNumber, token])

const isSelected = selectedOrder !== null

return (
<motion.article
layout
```

```
className={page-card ${isSelected ? 'selected' : ''}}
initial={{ opacity: 0, y: 12 }}
animate={{ opacity: 1, y: 0 }}
transition={{ duration: 0.24 }}
>
<button className="page-toggle" type="button" onClick={() => onToggle(pageNumber)}>
<span className="page-preview">
<canvas ref={canvasRef} aria-label={Preview of page ${pageNumber}} />
</span>
<span className="page-meta">
<span>Page {pageNumber}</span>
{isSelected && <strong>#{selectedOrder}</strong>}
</span>
</button>

{isSelected && (
<div className="page-actions" aria-label={Reorder page ${pageNumber}}>
<button type="button" onClick={() => onMove(pageNumber, -1)} title="Move earlier">
<ArrowUp size={16} />
</button>
<button type="button" onClick={() => onMove(pageNumber, 1)} title="Move later">
<ArrowDown size={16} />
</button>
</div>
)}
</motion.article>
)
}

function App() {
const [token, setToken] = useState(() => localStorage.getItem(STORAGE_KEYS.TOKEN) ?? '')
const [user, setUser] = useState<User | null>(null)
const [authMode, setAuthMode] = useState<AuthMode>('login')
const [authName, setAuthName] = useState('')
const [authEmail, setAuthEmail] = useState('')
const [authPassword, setAuthPassword] = useState('')
const [authOtp, setAuthOtp] = useState('')
const [devOtp, setDevOtp] = useState('')
const [isAuthLoading, setIsAuthLoading] = useState(false)
const [uploadedPdf, setUploadedPdf] = useState<UploadedPdf | null>(null)
const [selectedPages, setSelectedPages] = useState<number[]>([])
const [uploadProgress, setUploadProgress] = useState(0)
const [isDragging, setIsDragging] = useState(false)
const [isUploading, setIsUploading] = useState(false)
const [isExtracting, setIsExtracting] = useState(false)
const [extractedPdf, setExtractedPdf] = useState<ExtractedPdf | null>(null)
const [toast, setToast] = useState<ToastState | null>(null)

const showToast = (message: string, tone: ToastState['tone']) => {
setToast({ message, tone })
window.setTimeout(() => setToast(null), 3600)
}

useEffect(() => {
```

```
if (!token) {
return
}

axios.get<{ user: User }>(${API_BASE_URL}${API_ENDPOINTS.AUTH.ME}, {
headers: { [HTTP_HEADERS.AUTHORIZATION]: ${HTTP_HEADERS.BEARER} ${token} },
})
.then((response) => setUser(response.data.user))
.catch(() => {
localStorage.removeItem(STORAGE_KEYS.TOKEN)
setToken('')
setUser(null)
})
}, [token])

const saveSession = (nextToken: string, nextUser: User) => {
localStorage.setItem(STORAGE_KEYS.TOKEN, nextToken)
setToken(nextToken)
setUser(nextUser)
}

const handleSignup = async () => {
setIsAuthLoading(true)

try {
const response = await axios.post<{ message: string; user: User; devOtp?: string }>(
${API_BASE_URL}${API_ENDPOINTS.AUTH.SIGNUP},
{ name: authName, email: authEmail, password: authPassword },
)

setAuthMode('verify')
setDevOtp(response.data.devOtp ?? '')
showToast(response.data.message, 'success')
} catch (error) {
const message = axios.isAxiosError(error)
? error.response?.data?.error ?? UI_MESSAGES.SIGNUP_FAILED
: UI_MESSAGES.SIGNUP_FAILED
showToast(message, 'error')
} finally {
setIsAuthLoading(false)
}
}

const handleVerifyOtp = async () => {
setIsAuthLoading(true)

try {
const response = await axios.post<{ message: string; user: User; token: string }>(
${API_BASE_URL}${API_ENDPOINTS.AUTH.VERIFY_OTP},
{ email: authEmail, otp: authOtp },
)

saveSession(response.data.token, response.data.user)
showToast(response.data.message, 'success')
```

```
} catch (error) {
const message = axios.isAxiosError(error)
? error.response?.data?.error ?? UI_MESSAGES.OTP_VERIFICATION_FAILED
: UI_MESSAGES.OTP_VERIFICATION_FAILED
showToast(message, 'error')
} finally {
setIsAuthLoading(false)
}
}

const handleResendOtp = async () => {
setIsAuthLoading(true)

try {
const response = await axios.post<{ message: string; devOtp?: string }>(
${API_BASE_URL}${API_ENDPOINTS.AUTH.RESEND_OTP},
{ email: authEmail },
)

setDevOtp(response.data.devOtp ?? '')
showToast(response.data.message, 'success')
} catch (error) {
const message = axios.isAxiosError(error)
? error.response?.data?.error ?? UI_MESSAGES.RESEND_OTP_FAILED
: UI_MESSAGES.RESEND_OTP_FAILED
showToast(message, 'error')
} finally {
setIsAuthLoading(false)
}
}

const handleLogin = async () => {
setIsAuthLoading(true)

try {
const response = await axios.post<{ message: string; user: User; token: string }>(
${API_BASE_URL}${API_ENDPOINTS.AUTH.LOGIN},
{ email: authEmail, password: authPassword },
)

saveSession(response.data.token, response.data.user)
showToast(response.data.message, 'success')
} catch (error) {
const message = axios.isAxiosError(error)
? error.response?.data?.error ?? UI_MESSAGES.LOGIN_FAILED
: UI_MESSAGES.LOGIN_FAILED
showToast(message, 'error')
} finally {
setIsAuthLoading(false)
}
}

const logout = () => {
localStorage.removeItem(STORAGE_KEYS.TOKEN)
```

```
setToken('')
setUser(null)
setUploadedPdf(null)
setSelectedPages([])
setExtractedPdf(null)
}

const uploadPdf = (file: File) => {
if (file.type !== 'application/pdf' && !file.name.toLowerCase().endsWith('.pdf')) {
showToast(UI_MESSAGES.INVALID_PDF, 'error')
return
}

const formData = new FormData()
formData.append('pdf', file)

const request = new XMLHttpRequest()
request.open('POST', ${API_BASE_URL}${API_ENDPOINTS.PDFS.UPLOAD})
request.setRequestHeader(HTTP_HEADERS.AUTHORIZATION, ${HTTP_HEADERS.BEARER} ${token})
setIsUploading(true)
setUploadProgress(0)
setExtractedPdf(null)

request.upload.onprogress = (event) => {
if (event.lengthComputable) {
setUploadProgress(Math.round((event.loaded / event.total) * 100))
}
}

request.onload = () => {
setIsUploading(false)

try {
const response = JSON.parse(request.responseText)

if (request.status >= 400) {
showToast(response.error ?? UI_MESSAGES.UPLOAD_FAILED, 'error')
return
}

setUploadedPdf(response)
setSelectedPages([])
setUploadProgress(100)
showToast(UI_MESSAGES.PDF_UPLOADED, 'success')
} catch {
showToast(UI_MESSAGES.UPLOAD_UNEXPECTED_RESPONSE, 'error')
}
}

request.onerror = () => {
setIsUploading(false)
showToast(UI_MESSAGES.BACKEND_UNREACHABLE, 'error')
}
```

```
request.send(formData)
}

const handleFileInput = (event: ChangeEvent<HTMLInputElement>) => {
const file = event.target.files?.[0]

if (file) {
uploadPdf(file)
}
}

const handleDrop = (event: DragEvent<HTMLLabelElement>) => {
event.preventDefault()
setIsDragging(false)

const file = event.dataTransfer.files[0]

if (file) {
uploadPdf(file)
}
}

const togglePage = (pageNumber: number) => {
setSelectedPages((currentPages) => {
if (currentPages.includes(pageNumber)) {
return currentPages.filter((page) => page !== pageNumber)
}

return [...currentPages, pageNumber]
})
setExtractedPdf(null)
}

const movePage = (pageNumber: number, direction: -1 | 1) => {
setSelectedPages((currentPages) => {
const index = currentPages.indexOf(pageNumber)
const targetIndex = index + direction

if (index === -1 || targetIndex < 0 || targetIndex >= currentPages.length) {
return currentPages
}

const nextPages = [...currentPages]
nextPages[index] = currentPages[targetIndex]
nextPages[targetIndex] = pageNumber
return nextPages
})
}

const selectAllPages = () => {
if (!uploadedPdf) {
return
}
```

```
setSelectedPages(Array.from({ length: uploadedPdf.pageCount }, (_, index) => index + 1))
setExtractedPdf(null)
}

const resetWorkspace = () => {
setUploadedPdf(null)
setSelectedPages([])
setExtractedPdf(null)
setUploadProgress(0)
}

const extractPages = async () => {
if (!uploadedPdf || selectedPages.length === 0) {
showToast(UI_MESSAGES.SELECT_PAGE_FIRST, 'error')
return
}

setIsExtracting(true)

try {
const response = await axios.post<ExtractedPdf>(
${API_BASE_URL}${API_ENDPOINTS.PDFS.EXTRACT(uploadedPdf.id)},
{ pages: selectedPages },
{ headers: { [HTTP_HEADERS.AUTHORIZATION]: ${HTTP_HEADERS.BEARER} ${token} } },
)

setExtractedPdf(response.data)
showToast(UI_MESSAGES.EXTRACTION_SUCCESS, 'success')
} catch (error) {
const message = axios.isAxiosError(error)
? error.response?.data?.error ?? UI_MESSAGES.EXTRACTION_FAILED
: UI_MESSAGES.EXTRACTION_FAILED

showToast(message, 'error')
} finally {
setIsExtracting(false)
}
}

const previewUrl = uploadedPdf ? getAssetUrl(uploadedPdf.previewUrl) : ''
const downloadUrl = extractedPdf ? getAssetUrl(extractedPdf.downloadUrl) : ''
const selectedSummary = selectedPages.length > 0 ? selectedPages.join(', ') : 'No pages selected'

if (!user) {
return (
<ErrorBoundary>
<main className="app-shell auth-shell">
<section className="auth-card">
<div className="auth-brand">
<ShieldCheck size={38} />
<p className="eyebrow">Secure PDF Workspace</p>
<h1>{authMode === 'login' ? 'Login to continue.' : authMode === 'signup' ? 'Create your account.' : 'Verify your
email.'}</h1>
</div>
```

```
<div className="auth-tabs">
<button className={authMode === 'login' ? 'active' : ''} type="button" onClick={() => setAuthMode('login')}>
Login
</button>
<button className={authMode === 'signup' ? 'active' : ''} type="button" onClick={() => setAuthMode('signup')}>
Signup
</button>
</div>

{authMode === 'signup' && (
<label className="field">
<span>Name</span>
<input value={authName} onChange={(event) => setAuthName(event.target.value)} placeholder="Your name" />
</label>
)}

<label className="field">
<span>Email</span>
<input value={authEmail} onChange={(event) => setAuthEmail(event.target.value)} placeholder="you@example.com" />
</label>

{authMode !== 'verify' && (
<label className="field">
<span>Password</span>
<input type="password" value={authPassword} onChange={(event) => setAuthPassword(event.target.value)}
placeholder="At least 8 characters" />
</label>
)}

{authMode === 'verify' && (
<>
<label className="field">
<span>OTP</span>
<input value={authOtp} onChange={(event) => setAuthOtp(event.target.value)} placeholder="6 digit code" />
</label>
{devOtp && <p className="dev-otp">Development OTP: {devOtp}</p>}
</>
)}

<button
className="primary-button wide"
type="button"
disabled={isAuthLoading}
onClick={authMode === 'login' ? handleLogin : authMode === 'signup' ? handleSignup : handleVerifyOtp}
>
{isAuthLoading && <Loader2 className="spin" size={18} />}
{authMode === 'login' ? 'Login' : authMode === 'signup' ? 'Create account' : 'Verify account'}
</button>

{authMode === 'verify' && (
<button className="ghost-button wide" type="button" disabled={isAuthLoading} onClick={handleResendOtp}>
Resend OTP
```

```
</button>
)}
</section>

{toast && <div className={toast ${toast.tone}}>{toast.message}</div>}
</main>
</ErrorBoundary>
)
}

return (
<ErrorBoundary>
<main className="app-shell">
<section className="workspace-header">
<div>
<p className="eyebrow">PDF Page Extractor</p>
<h1>Select, reorder, and extract pages into a new PDF.</h1>
<p className="user-line">Signed in as {user.name} {user.email}</p>
</div>
<div className="header-actions">
{uploadedPdf && (
<button className="ghost-button" type="button" onClick={resetWorkspace}>
<RotateCcw size={17} />
```

New file

```
</button>
)}
<button className="ghost-button" type="button" onClick={logout}>
<LogOut size={17} />
```

Logout

```
</button>
</div>
</section>

<section className="uploader-panel">
<label
className={upload-zone ${isDragging ? 'dragging' : ''}}
onDragOver={(event) => {
event.preventDefault()
setIsDragging(true)
}}
onDragLeave={() => setIsDragging(false)}
onDrop={handleDrop}
>
<input type="file" accept="application/pdf,.pdf" onChange={handleFileInput} />
<UploadCloud size={34} />
<span>{isUploading ? 'Uploading your PDF...' : 'Drop a PDF here or choose a file'}</span>
<small>Only PDF files are accepted. Maximum backend limit is 50 MB.</small>
{isUploading && (
<span className="progress-track">
<span style={{ width: ${uploadProgress}% }} />
</span>
)}
</label>
```

```
<div className="file-status">
{uploadedPdf ? (
<>
<FileText size={22} />
<div>
<strong>{uploadedPdf.name}</strong>
<span>
{uploadedPdf.pageCount} pages {formatFileSize(uploadedPdf.size)}
</span>
</div>
</>
) : (
<>
<Scissors size={22} />
<div>
<strong>Ready for a PDF</strong>
<span>The preview grid appears after upload.</span>
</div>
</>
)}
</div>
</section>

{uploadedPdf ? (
<>
<section className="toolbar">
<div>
<strong>{selectedPages.length} selected</strong>
<span>{selectedSummary}</span>
</div>
<div className="toolbar-actions">
<button type="button" className="ghost-button" onClick={selectAllPages}>
<Check size={17} />
All
</button>
<button type="button" className="ghost-button" onClick={() => setSelectedPages([])}>
<X size={17} />
Clear
</button>
<button className="primary-button" type="button" onClick={extractPages} disabled={isExtracting}>
{isExtracting ? <Loader2 className="spin" size={18} /> : <Scissors size={18} />}
Extract
</button>
{extractedPdf && (
<a className="download-button" href={downloadUrl} download={extractedPdf.fileName}>
<Download size={18} />
Download
</a>
)}
</div>
</section>

<section className="page-grid" aria-label="PDF pages">
```

```
{Array.from({ length: uploadedPdf.pageCount }, (_, index) => {
const pageNumber = index + 1
const selectedIndex = selectedPages.indexOf(pageNumber)

return (
<PageCard
key={pageNumber}
pdfUrl={previewUrl}
token={token}
pageNumber={pageNumber}
selectedOrder={selectedIndex === -1 ? null : selectedIndex + 1}
onToggle={togglePage}
onMove={movePage}
/>
)
})}
</section>
</>
) : (
<section className="empty-state">
<FileText size={42} />
<h2>Upload a PDF to begin</h2>
<p>Every page will render as a thumbnail so you can choose the exact output order.</p>
</section>
)}

{toast && <div className={toast ${toast.tone}}>{toast.message}</div>}
</main>
</ErrorBoundary>
)
}

export default App
```

frontend/src/App.css

```
.app-shell {
width: min(1180px, calc(100% - 32px));
min-height: 100svh;
margin: 0 auto;
padding: 32px 0 48px;
}

.workspace-header {
display: flex;
align-items: flex-start;
justify-content: space-between;
gap: 24px;
margin-bottom: 24px;
}

.workspace-header h1 {
max-width: 820px;
```

```
}

.header-actions {
display: flex;
align-items: center;
justify-content: flex-end;
flex-wrap: wrap;
gap: 10px;
}

.user-line {
margin-top: 12px;
color: var(--text-muted);
}

.auth-shell {
display: grid;
place-items: center;
}

.auth-card {
width: min(520px, 100%);
display: grid;
gap: 16px;
border: 1px solid var(--border);
border-radius: 8px;
padding: 24px;
background: var(--glass);
box-shadow: var(--shadow);
backdrop-filter: blur(18px);
}

.auth-brand {
display: grid;
gap: 8px;
}

.auth-brand svg {
color: var(--cyan);
}

.auth-brand h1 {
font-size: clamp(2rem, 6vw, 3.4rem);
}

.auth-tabs {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 8px;
padding: 4px;
border: 1px solid var(--border);
border-radius: 8px;
background: rgba(255, 255, 255, 0.055);
}
```

```
.auth-tabs button {
border-color: transparent;
background: transparent;
}

.auth-tabs button.active {
background: rgba(117, 225, 255, 0.16);
color: var(--cyan);
}

.field {
display: grid;
gap: 7px;
color: var(--text-strong);
font-size: 0.9rem;
font-weight: 800;
}

.field input {
width: 100%;
min-height: 46px;
border: 1px solid var(--border);
border-radius: 8px;
padding: 0 13px;
outline: none;
color: var(--text-strong);
background: rgba(8, 12, 22, 0.76);
}

.field input:focus {
border-color: rgba(117, 225, 255, 0.72);
}

.dev-otp {
border: 1px solid rgba(116, 240, 181, 0.3);
border-radius: 8px;
padding: 10px 12px;
color: var(--mint);
background: rgba(116, 240, 181, 0.08);
font-size: 0.92rem;
}

.wide {
width: 100%;
}

.eyebrow {
margin: 0 0 10px;
color: var(--cyan);
font-size: 0.78rem;
font-weight: 800;
letter-spacing: 0.12em;
text-transform: uppercase;
```

```
}

.uploader-panel,
.toolbar,
.empty-state {
border: 1px solid var(--border);
background: var(--glass);
box-shadow: var(--shadow);
backdrop-filter: blur(18px);
}

.uploader-panel {
display: grid;
grid-template-columns: minmax(0, 1.4fr) minmax(260px, 0.6fr);
gap: 18px;
border-radius: 8px;
padding: 18px;
}

.upload-zone {
min-height: 190px;
display: grid;
place-items: center;
gap: 8px;
padding: 28px;
border: 1px dashed rgba(117, 225, 255, 0.45);
border-radius: 8px;
color: var(--text);
background: rgba(8, 12, 22, 0.72);
cursor: pointer;
transition: border-color 180ms ease, background 180ms ease, transform 180ms ease;
}

.upload-zone.dragging,
.upload-zone:hover {
border-color: var(--cyan);
background: rgba(17, 27, 46, 0.92);
transform: translateY(-1px);
}

.upload-zone input {
position: absolute;
width: 1px;
height: 1px;
opacity: 0;
pointer-events: none;
}

.upload-zone svg {
color: var(--cyan);
}

.upload-zone span {
color: var(--text-strong);
```

```
font-size: 1.08rem;
font-weight: 800;
}

.upload-zone small {
color: var(--text-muted);
}

.progress-track {
width: min(360px, 100%);
height: 8px;
overflow: hidden;
border-radius: 999px;
background: rgba(255, 255, 255, 0.09);
}

.progress-track span {
display: block;
height: 100%;
border-radius: inherit;
background: linear-gradient(90deg, var(--cyan), var(--pink));
}

.file-status {
min-height: 100%;
display: flex;
align-items: center;
gap: 14px;
padding: 20px;
border-radius: 8px;
background: rgba(255, 255, 255, 0.055);
}

.file-status svg {
flex: 0 0 auto;
color: var(--pink);
}

.file-status div {
min-width: 0;
}

.file-status strong,
.file-status span {
display: block;
}

.file-status strong {
overflow: hidden;
color: var(--text-strong);
font-size: 0.98rem;
text-overflow: ellipsis;
white-space: nowrap;
}
```

```
.file-status span {
margin-top: 4px;
color: var(--text-muted);
font-size: 0.88rem;
}

.toolbar {
position: sticky;
z-index: 10;
top: 14px;
display: flex;
align-items: center;
justify-content: space-between;
gap: 18px;
margin: 22px 0;
padding: 14px;
border-radius: 8px;
}

.toolbar strong,
.toolbar span {
display: block;
}

.toolbar strong {
color: var(--text-strong);
}

.toolbar span {
max-width: min(520px, 42vw);
overflow: hidden;
color: var(--text-muted);
font-size: 0.86rem;
text-overflow: ellipsis;
white-space: nowrap;
}

.toolbar-actions {
display: flex;
align-items: center;
flex-wrap: wrap;
justify-content: flex-end;
gap: 9px;
}

button,
.download-button {
min-height: 40px;
display: inline-flex;
align-items: center;
justify-content: center;
gap: 8px;
border-radius: 8px;
```

```
border: 1px solid transparent;
padding: 0 14px;
color: var(--text-strong);
font: inherit;
font-size: 0.9rem;
font-weight: 800;
text-decoration: none;
cursor: pointer;
transition: transform 160ms ease, border-color 160ms ease, background 160ms ease;
}

button:hover,
.download-button:hover {
transform: translateY(-1px);
}

button:disabled {
cursor: not-allowed;
opacity: 0.7;
}

.ghost-button {
background: rgba(255, 255, 255, 0.07);
border-color: var(--border);
}

.primary-button,
.download-button {
background: linear-gradient(135deg, var(--cyan), var(--pink));
color: #06101a;
}

.download-button {
background: linear-gradient(135deg, var(--mint), var(--cyan));
}

.page-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(190px, 1fr));
gap: 18px;
}

.page-card {
position: relative;
min-width: 0;
overflow: hidden;
border: 1px solid var(--border);
border-radius: 8px;
background: rgba(255, 255, 255, 0.055);
transition: border-color 180ms ease, background 180ms ease, transform 180ms ease;
}

.page-card.selected {
border-color: rgba(117, 225, 255, 0.8);
```

```
background: rgba(117, 225, 255, 0.1);
}

.page-toggle {
width: 100%;
height: 100%;
display: block;
padding: 10px;
border: 0;
border-radius: 0;
background: transparent;
text-align: left;
}

.page-preview {
aspect-ratio: 3 / 4;
display: grid;
place-items: center;
overflow: hidden;
border-radius: 6px;
background: #0a0d16;
}

.page-preview canvas {
width: 100%;
height: 100%;
object-fit: contain;
}

.page-meta {
display: flex;
align-items: center;
justify-content: space-between;
gap: 8px;
padding-top: 10px;
color: var(--text-strong);
}

.page-meta span {
font-weight: 800;
}

.page-meta strong {
min-width: 32px;
min-height: 28px;
display: inline-grid;
place-items: center;
border-radius: 999px;
background: var(--cyan);
color: #06101a;
font-size: 0.8rem;
}

.page-actions {
```

```
position: absolute;
top: 14px;
right: 14px;
display: flex;
gap: 6px;
}

.page-actions button {
width: 34px;
min-height: 34px;
padding: 0;
background: rgba(6, 10, 18, 0.8);
border-color: rgba(255, 255, 255, 0.16);
}

.empty-state {
display: grid;
place-items: center;
min-height: 280px;
margin-top: 24px;
padding: 42px 22px;
border-radius: 8px;
text-align: center;
}

.empty-state svg {
color: var(--cyan);
}

.empty-state h2,
.empty-state h1 {
margin: 14px 0 8px;
}

.empty-state p {
max-width: 500px;
color: var(--text-muted);
}

.toast {
position: fixed;
right: 24px;
bottom: 24px;
max-width: min(360px, calc(100% - 32px));
z-index: 30;
border: 1px solid var(--border);
border-radius: 8px;
padding: 14px 16px;
color: var(--text-strong);
box-shadow: var(--shadow);
backdrop-filter: blur(16px);
}

.toast.success {
```

```
background: rgba(34, 197, 94, 0.18);
border-color: rgba(34, 197, 94, 0.38);
}

.toast.error {
background: rgba(251, 113, 133, 0.18);
border-color: rgba(251, 113, 133, 0.38);
}

.spin {
animation: spin 850ms linear infinite;
}

@keyframes spin {
to {
transform: rotate(360deg);
}
}

@media (max-width: 820px) {
.app-shell {
width: min(100% - 22px, 1180px);
padding-top: 18px;
}

.workspace-header,
.uploader-panel,
.toolbar {
grid-template-columns: 1fr;
flex-direction: column;
align-items: stretch;
}

.toolbar span {
max-width: 100%;
}

.toolbar-actions {
justify-content: stretch;
}

.toolbar-actions button,
.toolbar-actions a {
flex: 1 1 132px;
}

.page-grid {
grid-template-columns: repeat(auto-fill, minmax(145px, 1fr));
gap: 12px;
}
}
```

frontend/src/index.css

```
:root {
--bg: #080a12;
--surface: #111827;
--glass: rgba(16, 24, 39, 0.78);
--border: rgba(255, 255, 255, 0.12);
--text: #d7deea;
--text-muted: #8f9db4;
--text-strong: #f8fbff;
--cyan: #75e1ff;
--pink: #ff79c6;
--mint: #74f0b5;
--shadow: 0 22px 80px rgba(0, 0, 0, 0.35);
--font: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
color: var(--text);
background:
linear-gradient(135deg, rgba(117, 225, 255, 0.12) 0%, transparent 32%),
linear-gradient(225deg, rgba(255, 121, 198, 0.1) 0%, transparent 34%),
linear-gradient(135deg, #080a12 0%, #111525 48%, #0b1721 100%);
font-family: var(--font);
font-synthesis: none;
line-height: 1.5;
text-rendering: optimizeLegibility;
-webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale;
}

- {
box-sizing: border-box;
}

html {
min-width: 320px;
min-height: 100%;
}

body {
min-width: 320px;
min-height: 100svh;
margin: 0;
}

body::before {
position: fixed;
inset: 0;
z-index: -1;
content: "";
background-image:
linear-gradient(rgba(255, 255, 255, 0.035) 1px, transparent 1px),
linear-gradient(90deg, rgba(255, 255, 255, 0.035) 1px, transparent 1px);
background-size: 48px 48px;
mask-image: linear-gradient(to bottom, black, transparent 78%);
}
```

```
h1,
h2,
p {
margin: 0;
}

h1 {
color: var(--text-strong);
font-size: clamp(2rem, 4vw, 4.3rem);
line-height: 0.98;
letter-spacing: 0;
}

h2 {
color: var(--text-strong);
font-size: 1.45rem;
line-height: 1.2;
letter-spacing: 0;
}

button,
input {
font: inherit;
}

::selection {
background: rgba(117, 225, 255, 0.28);
}
```

frontend/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>PDF Page Extractor</title>
</head>
<body>
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
</body>
</html>
```

README.md

PDF Page Extractor

A full-stack PDF manipulation app with authentication. Users sign up, verify email OTP, log in with JWT, upload PDFs, preview pages, select/reorder pages, and download a newly generated PDF.

Tech Stack

- Frontend: React, Vite, TypeScript, Vanilla CSS, Framer Motion, PDF.js, Lucide React
- Backend: Node.js, Express, TypeScript, Multer, PDF-lib, Node Cron
- Auth: JWT, bcrypt password hashing, OTP email verification with Nodemailer
- Storage: Local file system plus JSON records for users and PDF ownership
- Tests: Node built-in test runner for backend PDF extraction logic

Architecture

Current SOLID, DTO, and Mapping Additions

The backend now keeps request validation and response/storage transformation out of the services:

- backend/src/controllers/authController.ts and backend/src/controllers/pdfController.ts are class-based controllers. Their public methods orchestrate HTTP request/response flow.
- backend/src/app.ts uses AppFactory and ErrorHandlerMiddleware classes for Express setup and global error handling.
- backend/src/routes/authRoutes.ts and backend/src/routes/pdfRoutes.ts use route classes for endpoint registration.
- backend/src/middleware/authenticate.ts, backend/src/config/multer.ts, and backend/src/utils/cleanup.ts use classes for auth middleware, upload configuration, and cleanup scheduling.
- backend/src/dtos/authDtos.ts validates and normalizes signup, login, verify OTP, and resend OTP request bodies.
- backend/src/dtos/pdfDtos.ts validates the extraction request and maps one-based UI page numbers to zero-based pdf-lib page indexes.
- backend/src/mappers/userMapper.ts converts UserRecord into PublicUser.
- backend/src/mappers/pdfMapper.ts converts upload metadata into PdfRecord and creates upload/extract response **DTOs.**
- backend/test/dtoValidation.test.ts covers DTO normalization, OTP validation, page mapping, and invalid pages payloads.

Layer responsibilities:

Route

-> Route class
-> Controller class method
-> DTO validator
-> Service
-> Repository / PDF-lib / File system
-> Mapper
-> Response DTO

SOLID notes:

- Single Responsibility: each layer has one reason to change.
- Open/Closed: new DTOs or mappers can be added without rewriting PDF extraction or auth logic.
- Liskov Substitution: validators/mappers/services can be swapped with equivalent test doubles.
- Interface Segregation: controllers use only the small methods they need.
- Dependency Inversion: controllers receive services, validators, repositories, and mappers through constructor parameters.

The backend follows a clean layered flow:

- routes: maps REST endpoints to controller functions.
- controllers: validates HTTP input and shapes HTTP responses.

- services: contains auth, email, and PDF business logic.
- constants: keeps route paths, status codes, messages, storage names, and limits in one place.
- repositories: stores users and PDF ownership records in JSON files.
- middleware: verifies JWT tokens before protected routes.
- config: keeps upload/storage middleware setup.
- utils: contains background cleanup jobs.

Auth flow:

- User signs up with name, email, and password.
- Backend hashes the password with bcrypt.
- Backend generates a 6-digit OTP, hashes it, stores expiry/attempt data, and sends it by email.
- User verifies OTP.
- Backend marks account verified and returns a JWT token.
- Frontend stores the token in localStorage.
- Protected PDF APIs require Authorization: Bearer <token>.
- Uploaded PDFs are saved with the logged-in user's userId.

Setup

Open two terminals.

Backend:

```
cd backend
npm install
copy .env.example .env
npm run dev
```

Frontend:

```
cd frontend
npm install
copy .env.example .env
npm run dev
```

Default URLs:

- Frontend: http://localhost:5173
- Backend: http://localhost:5000

Production-style backend run:

```
cd backend
npm run build
npm start
```

Environment Variables

Backend .env.example:

```
PORT=5000
```

```
NODE_ENV=development
UPLOAD_DIR=uploads
OUTPUT_DIR=outputs
CLEANUP_INTERVAL_MINUTES=60
JWT_SECRET=replace-with-a-long-random-secret
JWT_EXPIRES_IN=7d
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
SMTP_FROM=
```

If SMTP values are empty in development, the backend logs the OTP to the console and returns devOtp in the signup/resend response. In production, configure SMTP and never expose OTPs in responses.

Frontend .env.example:

```
VITE_API_URL=http://localhost:5000
```

API Endpoints

Auth

POST /api/auth/signup

- Body: { "name": "User", "email": "user@example.com", "password": "password123" }
- Creates an unverified user and sends OTP.

POST /api/auth/verify-otp

- Body: { "email": "user@example.com", "otp": "123456" }
- Verifies account and returns JWT token.

POST /api/auth/resend-otp

- Body: { "email": "user@example.com" }
- Sends a new OTP after cooldown.

POST /api/auth/login

- Body: { "email": "user@example.com", "password": "password123" }
- Returns JWT token for verified users.

GET /api/auth/me

- Header: Authorization: Bearer <token>
- Returns current user.

PDFs

POST /api/pdfs/upload

- Header: Authorization: Bearer <token>

- Form field: pdf
- Accepts only PDF files
- Returns PDF id, original name, size, page count, and preview URL

GET /api/pdfs/:id

- Header: Authorization: Bearer <token>
- Serves the uploaded PDF inline for frontend preview rendering

POST /api/pdfs/:id/extract

- Header: Authorization: Bearer <token>
- JSON body: { "pages": [3, 1, 2] }
- Page numbers are one-based and order matters
- Returns a generated PDF download URL

Tests

cd backend
npm test

The tests verify that selected pages are extracted and invalid page ranges are rejected.

Screenshots

Upload state:

Deployment Notes

For a live version, deploy the backend to a Node host and the frontend to a static host. Set JWT_SECRET, SMTP values, and VITE_API_URL before building/deploying.